

Chapter 9. Embedded Binary Information

Binary information is machine-readable information embedded in a binary by the SDK (or other development tools) such that it can be read by `picotool` or other tooling.

9.1. Basic information

This information is really handy when you pick up a Pico-series device and don't know what is on it!

Basic information includes

- program name
- program description
- program version string
- program build date
- program url
- program end address
- program features, this is a list built from individual strings in the binary, that can be displayed (e.g. we will have one for UART stdio and one for USB stdio) in the SDK
- build attributes, this is a similar list of strings, for things pertaining to the binary itself (e.g. Debug Build)

9.2. Pins

This is certainly handy when you have an executable called `hello_serial.elf` but you forgot what Raspberry Pi microcontroller-based board it was built for, as different boards may have different pins broken out.

Static (fixed) pin assignments can be recorded in the binary in very compact form:

```
$ picotool info --pins sprite_demo.elf
File sprite_demo.elf:

Fixed Pin Information
0-4:    Red 0-4
6-10:   Green 0-4
11-15:  Blue 0-4
16:     HSync
17:     VSync
18:     Display Enable
19:     Pixel Clock
20:     UART1 TX
21:     UART1 RX
```

9.3. Full Information

Full information is available with the `-a` option:

```
$ picotool info -a i2c_bus_scan.elf
File i2c_bus_scan.elf:

Program Information
name:          i2c_bus_scan
web site:      https://github.com/raspberrypi/pico-examples/tree/HEAD/i2c/bus_scan
features:      UART stdin / stdout
binary start:  0x10000000
binary end:     0x10004c74

Fixed Pin Information
0:  UART0 TX
1:  UART0 RX
4:  I2C0 SDA
5:  I2C0 SCL

Build Information
sdk version:    2.0.0-develop
pico_board:     pico
build date:     Aug  1 2024
build attributes: Debug
```

9.4. Including Binary Information

Binary information is declared in the program by macros; for the following example:

```
$ picotool info --pins sprite_demo.elf
File sprite_demo.elf:

Fixed Pin Information
0-4:   Red 0-4
6-10:  Green 0-4
11-15: Blue 0-4
16:    HSync
17:    VSync
18:    Display Enable
19:    Pixel Clock
20:    UART1 TX
21:    UART1 RX
```

There is one line in the `setup_default_uart` function:

```
bi_decl_if_func_used(bi_2pins_with_func(PICO_DEFAULT_UART_RX_PIN, PICO_DEFAULT_UART_TX_PIN,
GPIO_FUNC_UART));
```

The two pin numbers, and the function UART are stored, then decoded to their actual function names (UART1 TX etc) by picotool. The `bi_decl_if_func_used` makes sure the binary information is only included if the containing function is called.

Equally, the video code contains a few lines like this:

```
bi_decl_if_func_used(bi_pin_mask_with_name(0x1f << (PICO_SCANVIDEO_COLOR_PIN_BASE +
PICO_SCANVIDEO_DPI_PIXEL_RSHIFT), "Red 0-4"));
```

The macros are designed to waste as little space as possible, but you can turn everything off with preprocessor var `PICO_NO_BINARY_INFO=1`. Additionally, any SDK code that inserts binary info can be separately excluded by its own preprocessor var.

To add your own binary info, you need:

```
#include "pico/binary_info.h"
```

There are a bunch of `bi_` macros in the headers

```
#define bi_binary_end(end)
#define bi_program_name(name)
#define bi_program_description(description)
#define bi_program_version_string(version_string)
#define bi_program_build_date_string(date_string)
#define bi_program_url(url)
#define bi_program_feature(feature)
#define bi_program_build_attribute(attr)
#define bi_1pin_with_func(p0, func)
#define bi_2pins_with_func(p0, p1, func)
#define bi_3pins_with_func(p0, p1, p2, func)
#define bi_4pins_with_func(p0, p1, p2, p3, func)
#define bi_5pins_with_func(p0, p1, p2, p3, p4, func)
#define bi_pin_range_with_func(plo, phi, func)
#define bi_pin_mask_with_name(pmask, label)
#define bi_pin_mask_with_names(pmask, label)
#define bi_1pin_with_name(p0, name)
#define bi_2pins_with_names(p0, name0, p1, name1)
#define bi_3pins_with_names(p0, name0, p1, name1, p2, name2)
#define bi_4pins_with_names(p0, name0, p1, name1, p2, name2, p3, name3)
```

which make use of underlying macros, e.g.

```
#define bi_program_url(url) bi_string(BINARY_INFO_TAG_RASPBERRY_PI, BINARY_INFO_ID_RP_PROGRAM_URL,
url)
```

You then either use `bi_decl(bi_blah(...))` for unconditional inclusion of the binary info `blah`, or `bi_decl_if_func_used(bi_blah(...))` for binary information that may be stripped if the enclosing function is not included in the binary by the linker (think `--gc-sections`).

For example,

```
1 #include <stdio.h>
2 #include "pico/stdlib.h"
3 #include "hardware/gpio.h"
4 #include "pico/binary_info.h"
5
6 const uint LED_PIN = 25;
7
8 int main() {
```

```

9
10  bi_decl(bi_program_description("This is a test binary.));
11  bi_decl(bi_1pin_with_name(LED_PIN, "On-board LED"));
12
13  setup_default_uart();
14  gpio_set_function(LED_PIN, GPIO_FUNC_PROC);
15  gpio_set_dir(LED_PIN, GPIO_OUT);
16  while (1) {
17      gpio_put(LED_PIN, 0);
18      sleep_ms(250);
19      gpio_put(LED_PIN, 1);
20      puts("Hello World\n");
21      sleep_ms(1000);
22  }
23 }

```

when queried with `picotool`,

```

$ sudo picotool info -a test.uf2
File test.uf2:

Program Information
name:          test
description:    This is a test binary.
features:       stdout to UART
binary start:  0x10000000
binary end:     0x100031f8

Fixed Pin Information
0:  UART0 TX
1:  UART0 RX
25: On-board LED

Build Information
build date:  Jan  4 2021

```

shows our information strings in the output.

9.5. Setting Common Information from CMake

You can also set fields directly from your project's CMake file, e.g.,

```

pico_set_program_name(foo "not foo") ❶
pico_set_program_description(foo "this is a foo")
pico_set_program_version_string(foo "0.00001a")
pico_set_program_url(foo "www.plinth.com/foo")

```

1. The name "foo" would be the default.

i NOTE

All of these are passed as command line arguments to the compilation, so if you plan to use quotes, newlines etc. you may have better luck defining it using `bi_decl` in the code.